Deep Learning Application in Data Science

Neural Network Classification of Breast Cancer Diagnostic Data

Dataset: Breast Cancer Wisconsin (Diagnostic) Dataset — UCI Machine Learning Repository

Week 5 Task | Virtual Data Science with Python Trainee

Yuva Intern — NSDC

Submitted by: Rajkumar Yadav

1. Problem Statement

This report applies deep learning to the problem built up over the previous four weeks: predicting whether a tumor is **malignant or benign** from 30 diagnostic measurements in the Breast Cancer Wisconsin (Diagnostic) dataset. This is a **binary classification** task. In Week 4, Logistic Regression was the best of four classical models and reached an F1-score of 0.9639 on a held-out test set. This week a neural network is designed, trained and evaluated on exactly the same data and the same train/test split, so the two approaches can be compared fairly.

A note on the choice of dataset. With 569 rows of tabular data, this is not a setting where deep learning is expected to shine — images, text and very large tables are. It was chosen on purpose: it allows a direct comparison with Week 4, and it makes overfitting a real, observable challenge rather than a textbook one. The report therefore asks two questions: can a well-designed network be trained reliably on a small dataset, and does it justify its extra complexity over a simple baseline?

The model is built with **TensorFlow 2 / Keras**. All code, figures and results in this report come from the script `deep_learning_model.py` in the project repository.

2. Data Preparation

The input is the cleaned (unscaled) dataset produced by the Week 1 pipeline. The data was split exactly as in Week 4 (80/20, stratified, `random_state=42`). A further validation set was carved out of the training portion, because a neural network needs a validation signal during training to decide when to stop.

Subset	Samples	Purpose
Training	364	Fit the network weights
Validation	91	Early stopping and learning curves
Test	114 (42 malignant)	Touched once, for the final evaluation only

Features were standardized with `StandardScaler`, fitted on the non-test rows only and then applied to the test set. This matters more for neural networks than for many other models: gradient-based training is sensitive to feature scale, and unscaled inputs such as `mean area`

(hundreds) next to `mean smoothness` (about 0.1) would make optimization slow and unstable. Week 4's ablation test found that four engineered features did not improve performance, so the original 30 features are used unchanged — a neural network can also learn feature interactions on its own.

Code:

```
X_train, X_test, y_train, y_test = train_test_split(
    X.values, y, test_size=0.2, stratify=y, random_state=42)  # same as Week 4
scaler = StandardScaler().fit(X_train)       # fit on training data only
X_train_s, X_test_s = scaler.transform(X_train), scaler.transform(X_test)

X_tr, X_val, y_tr, y_val = train_test_split(
    X_train_s, y_train, test_size=0.2, stratify=y_train, random_state=42)
```

Output:

```
Train: 364  Validation: 91  Test: 114  Features: 30
```

3. Network Architecture and Design Justification

The chosen network is a small fully connected (multilayer perceptron) classifier with two hidden layers. How it was chosen is described in Section 4; its structure is shown below.

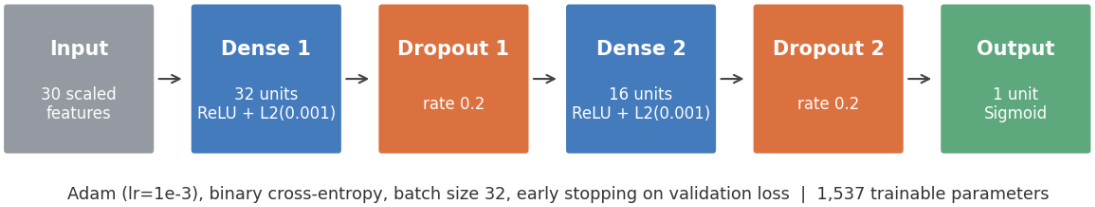


Figure 1: Architecture of the final network (1,537 trainable parameters)

Layer	Output shape	Parameters
Input (30 scaled features)	30	0
Dense 1 (ReLU, L2)	32	992
Dropout 1 (rate 0.2)	32	0
Dense 2 (ReLU, L2)	16	528
Dropout 2 (rate 0.2)	16	0
Output Dense (Sigmoid)	1	17
Total		**1,537**

Each design decision was made for a stated reason:

Decision	Choice	Justification
Hidden layers	2 layers (32 → 16 units)	A funnel shape compresses 30 inputs into progressively more abstract features. More depth is hard to justify with 364 training samples.
Hidden activation	ReLU	Cheap, and avoids the vanishing-gradient problem of sigmoid/tanh in hidden layers.

Decision	Choice	Justification
Output layer	1 unit, Sigmoid	Produces a probability of malignancy between 0 and 1 for a binary target.
Loss function	Binary cross-entropy	The natural loss for a probability output; it penalizes confident wrong predictions heavily.
Optimizer	Adam, learning rate 1e-3	Adaptive step sizes work well without extensive tuning; 1e-3 is a robust default.
Dropout	0.2 after each hidden layer	Randomly switches off units during training so the network cannot rely on any single unit.
L2 regularization	1e-3 on weights	Penalizes large weights, which discourages the network from memorizing training samples.
Early stopping	Patience 15, restore best weights	Stops training when validation loss stops improving and returns the best version of the model.
Batch size	32	Standard choice; gives about 12 weight updates per epoch on 364 samples.

Code:

```
def build_model(hidden=(32, 16), dropout=0.3, l2=1e-3, lr=1e-3, n_features=30):
    reg = regularizers.l2(l2) if l2 > 0 else None
    model = keras.Sequential([layers.Input(shape=(n_features,))])
    for units in hidden:
        model.add(layers.Dense(units, activation='relu', kernel_regularizer=reg))
        if dropout > 0:
            model.add(layers.Dropout(dropout))
    model.add(layers.Dense(1, activation='sigmoid'))
    model.compile(optimizer=keras.optimizers.Adam(lr),
                  loss='binary_crossentropy', metrics=['accuracy'])
    return model
```

4. Training Process

4.1 Demonstrating overfitting first

Before settling on a design, a deliberately oversized network (128 → 128 → 64 units, **28,801 parameters**) was trained for 200 epochs with no dropout, no L2 penalty and no early stopping. With 364 training samples, this network has roughly 80 parameters for every sample, so memorization is expected.

Code:

```
set_seed()
big = build_model(hidden=(128, 128, 64), dropout=0.0, l2=0.0)
hist_big = big.fit(X_tr, y_tr, validation_data=(X_val, y_val),
                   epochs=200, batch_size=32, verbose=0)
```

Output:

```
Best validation loss: 0.0636 at epoch 7
Final training loss:  4.7e-6   (training accuracy 100.0%)
Final validation loss: 0.1655  (validation accuracy 96.7%)
```

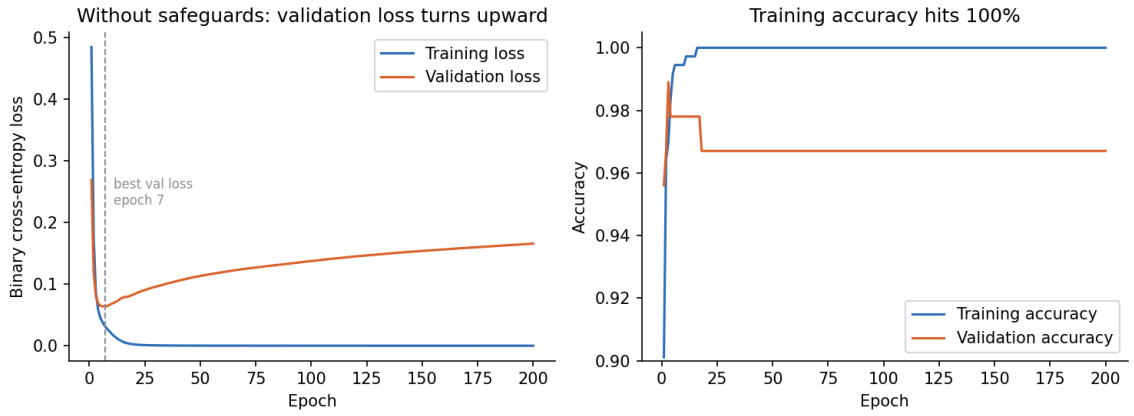


Figure 2: Overfitting in an unregularized network — validation loss rises after epoch 7

The result is a clear and instructive case of overfitting. Training loss falls to almost zero, while validation loss reaches its minimum at epoch 7 and then climbs steadily to **2.6 times** its best value. Notably, validation accuracy only slipped from about 98% to 96.7%. The damage shows up mainly in the loss: the network becomes over-confident, assigning extreme probabilities to samples it gets wrong. This is why validation loss, not accuracy, is used to monitor training.

4.2 Hyperparameter search

To choose the architecture and regularization strength objectively, twelve configurations were compared with **5-fold stratified cross-validation on the training data only** (the test set was not used). Three hyperparameters were searched: hidden layer sizes (16-8, 32-16, 64-32), dropout rate (0.2, 0.4) and L2 strength (1e-4, 1e-3). To avoid leakage, the scaler is fitted inside each fold, and early stopping uses a validation slice taken from each fold's training portion.

Code:

```
def cv_f1(config, X_raw, y_raw, n_splits=5):
    skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=42)
    scores = []
    for tr_idx, va_idx in skf.split(X_raw, y_raw):
        sc = StandardScaler().fit(X_raw[tr_idx])   # fit inside the fold
        Xa, Xb = sc.transform(X_raw[tr_idx]), sc.transform(X_raw[va_idx])
        Xa2, Xi, ya2, yi = train_test_split(Xa, y_raw[tr_idx],
            test_size=0.15, stratify=y_raw[tr_idx], random_state=42)
        set_seed(); m = build_model(**config)
        m.fit(Xa2, ya2, validation_data=(Xi, yi), epochs=150, batch_size=32,
              callbacks=[early_stop()], verbose=0)
        pred = (m.predict(Xb, verbose=0).ravel() >= 0.5).astype(int)
        scores.append(f1_score(y_raw[va_idx], pred))
    return np.mean(scores), np.std(scores)
```

Hidden layers	Dropout	L2	CV F1 (mean ± std)
32 → 16 ★ chosen	0.2	0.001	0.9735 ± 0.0111
32 → 16	0.4	0.001	0.9735 ± 0.0111
64 → 32	0.2	0.001	0.9733 ± 0.0116
32 → 16	0.2	0.0001	0.9706 ± 0.0095
32 → 16	0.4	0.0001	0.9706 ± 0.0095

Hidden layers	Dropout	L2	CV F1 (mean ± std)
64 → 32	0.4	0.0001	0.9703 ± 0.0167
64 → 32	0.4	0.001	0.9674 ± 0.0176
64 → 32	0.2	0.0001	0.9646 ± 0.0204
16 → 8	0.4	0.0001	0.9550 ± 0.0217
16 → 8	0.4	0.001	0.9550 ± 0.0217
16 → 8	0.2	0.0001	0.9523 ± 0.0113
16 → 8	0.2	0.001	0.9523 ± 0.0113

The best configuration was **32 → 16 units, dropout 0.2, L2 = 1e-3** with a cross-validated F1 of **0.9735**. Under the identical cross-validation protocol, Logistic Regression (C = 10) scored 0.9581 ± 0.0183. Two honest caveats apply. First, several configurations tied or came within a fraction of a standard deviation of the winner (for example the 64 → 32 network at 0.9733), so the ranking among the top few is not statistically meaningful; the smaller network was kept because it reaches the same performance with fewer parameters (1,537 versus 4,097). Second, within the ranges tried, dropout and L2 had very little effect, while the layer size mattered more: the smallest network (16 → 8) was clearly worse at about 0.952–0.955.

4.3 Training the final model

Code:

```
set_seed()
model = build_model(hidden=(32, 16), dropout=0.2, l2=1e-3)
hist = model.fit(X_tr, y_tr, validation_data=(X_val, y_val),
                 epochs=300, batch_size=32,
                 callbacks=[EarlyStopping(monitor='val_loss',
                            patience=15, restore_best_weights=True)],
                 verbose=0)
```

Output:

```
Stopped after 53 epochs; best epoch 38
```

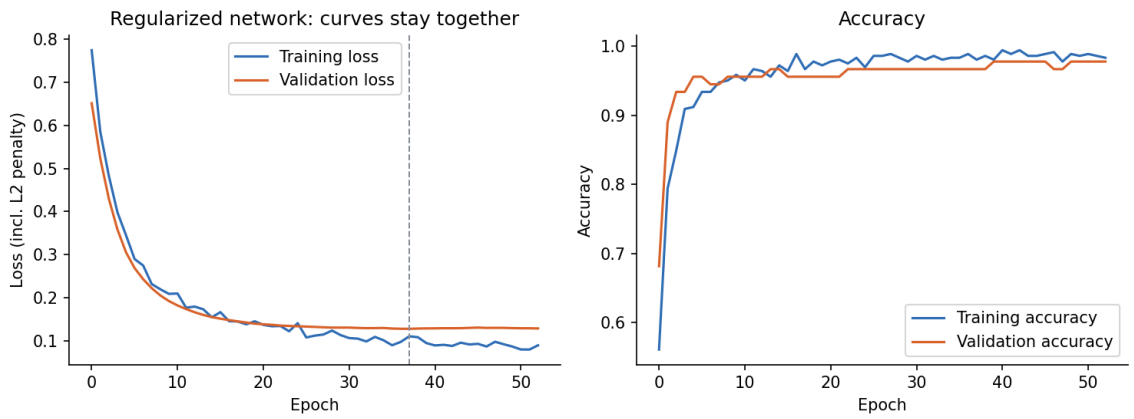


Figure 3: Learning curves of the final regularized network

Training stopped automatically after 53 epochs and the weights from epoch 38 were restored. In contrast with Figure 2, the training and validation curves stay close together and flatten out instead of diverging: training accuracy ends around 98% against about 97.8% on validation. The two safeguards — regularization and early stopping — are what keep this

network, which still has more parameters (1,537) than training samples (364), from memorizing the data.

5. Evaluation

The final network was evaluated once on the 114 held-out test patients and compared with Week 4's Logistic Regression (C = 10) trained on the same split.

Metric	Neural network	Logistic Regression (Week 4)	Difference
Accuracy	98.25%	97.37%	+0.88 pts
Precision	100.00%	97.56%	+2.44 pts
Recall	95.24%	95.24%	0.00 pts
F1-score	97.56%	96.39%	+1.18 pts
ROC-AUC	0.9931	0.9848	+0.0083

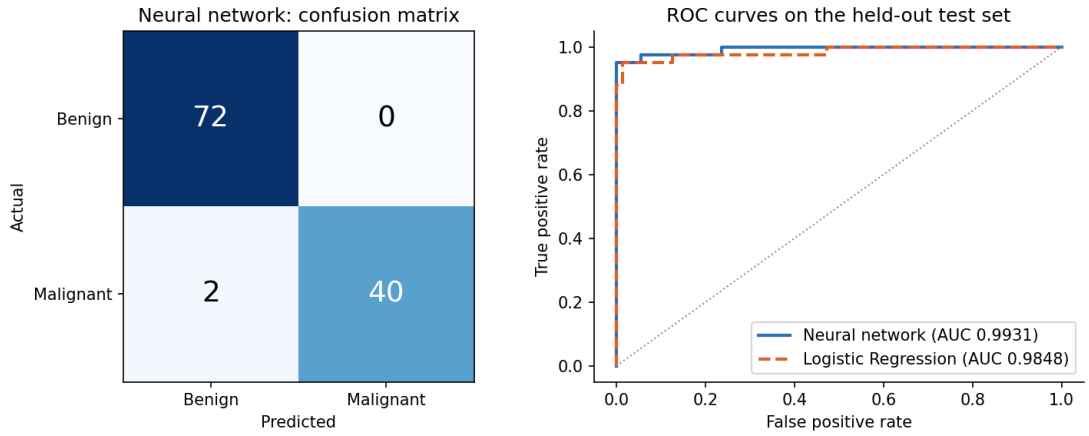


Figure 4: Confusion matrix of the neural network and ROC curves for both models

The network made **2 errors in 114 patients**, both false negatives (malignant tumors predicted as benign), and no false positives. Logistic Regression made 3 errors. The neural network's only advantage is that it removed one false positive; both models missed the same two malignant patients, so recall is identical at 95.24% (40 of 42).

How much weight should this difference carry? Very little. A single patient separates the two models. Because neural network training is random, the whole procedure was repeated with five different random seeds, keeping the data split fixed:

Metric	Mean over 5 seeds	Std. deviation
Accuracy	98.42%	1.14 pts
Precision	99.05%	2.13 pts
Recall	96.67%	1.30 pts
F1-score	97.84%	1.55 pts
ROC-AUC	0.9945	0.0015

Test F1 across the five seeds ranged from 0.9524 to 0.9880. Logistic Regression's 0.9639 falls inside that range, so on this dataset the neural network is **at least as good as the simple baseline, but not demonstrably better**. The consistently higher ROC-AUC (about 0.994 versus 0.985) suggests the network ranks patients slightly better, but with only 42 malignant test cases even that should be treated as suggestive rather than proven.

A methodological note: Week 4 reported a cross-validated F1 of 0.9641 for Logistic Regression, while the figure under this week's stricter protocol (scaler fitted inside each fold) is 0.9581. The small gap is a mild leakage effect in the earlier setup. It does not change any conclusion, and the comparison in this report uses the stricter protocol for both models.

6. Challenges Encountered

Challenge	What happened	How it was addressed
Overfitting	The unregularized network reached 100% training accuracy while validation loss rose to 2.6× its minimum (Section 4.1).	Dropout, L2 weight penalty, a compact architecture, and early stopping with best-weight restoration.
Very small dataset	Only 114 test samples (42 malignant), so one patient moves F1 by about 1 point. Results also vary from seed to seed (F1 std ≈ 0.015).	Cross-validation for model selection, five-seed repetition, and cautious claims about differences smaller than the noise.
Fair comparison with Week 4	Scaling before cross-validation slightly inflated the earlier baseline.	Same split as Week 4; scaler fitted inside every fold; both models scored under the same protocol.
Computational cost	The search needed 60 training runs (12 configurations × 5 folds) and took about 15 minutes on a CPU-only machine.	Kept the grid small, fixed the learning rate and batch size, and kept the networks compact.
Hyperparameter ties	Several configurations scored identically or within noise of each other.	Preferred the simpler model among near-ties instead of over-reading small differences.
Reproducibility	Neural network training is stochastic.	Fixed random seeds for data splits and model initialization; results reported over five seeds.

7. Critical Analysis of Results

7.1 Where the network still fails

Week 4 identified two malignant patients that Logistic Regression classified as benign with very high confidence. The neural network makes the same two mistakes:

Test row	Actual	NN probability of malignancy	Logistic Regression probability	MC-dropout uncertainty (std)
16	Malignant	0.0107	0.0191	0.0427
87	Malignant	0.1872	0.0001	0.3414

Two different models, trained by different methods, failing on the same two patients is informative: these are likely genuinely atypical cases whose measurements look benign, not

artifacts of one model's quirks. A different model type is unlikely to fix them; different information (for example imaging or additional tests) probably would.

7.2 Can the network tell when it is unsure?

A weakness noted in Week 4 was that errors came with no warning signal. To test whether a neural network can do better, **Monte Carlo dropout** was used: dropout is left switched on at prediction time and each patient is scored 200 times, and the spread of the predictions is taken as a measure of uncertainty.

Code:

```
mc = np.stack([model(X_test_s, training=True).numpy().ravel()
               for _ in range(200)])
mc_mean, mc_std = mc.mean(0), mc.std(0)
```

Output:

```
Mean uncertainty, correct predictions: 0.0334
Mean uncertainty, errors:              0.1921
Review rule (std > 0.0960): flags 13 of 114 patients, catches 1 of 2 errors
```

On average, the errors are much more uncertain than the correct predictions (0.1921 versus 0.0334). But the result is mixed in practice: one of the two errors (row 87, uncertainty 0.3414) is flagged, while the other (row 16, uncertainty 0.0427) looks as certain as a typical correct case and slips through. With only two errors this is an anecdote, not a proof. The honest conclusion is that dropout-based uncertainty is a useful but incomplete warning signal.

7.3 What did the network learn?

Permutation importance measures how much the test F1-score drops when one feature's values are shuffled. It was used for interpretation only; no modelling decision depends on it.

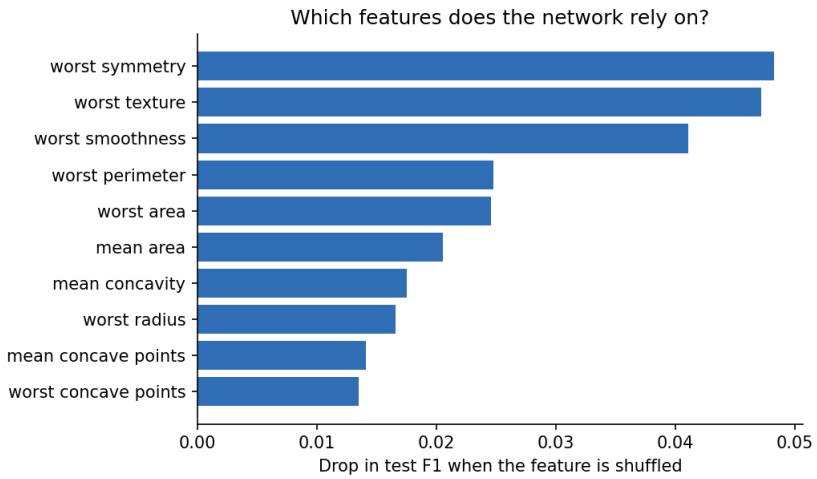


Figure 5: Permutation importance of the ten most influential features

The network relies most on **worst symmetry**, **worst texture** and **worst smoothness**. This differs from Week 2, where `worst concave points` and `worst perimeter` had the largest effect sizes; here `worst concave points` ranks only tenth. A plausible explanation — not tested here — is that many size and shape features are highly correlated (visible in Week 2's correlation heatmap), so shuffling one of them costs little because correlated substitutes remain, whereas texture, symmetry and smoothness carry information that no other feature

supplies. The importance values are small (at most about 0.05) and computed on only 114 samples, so the ranking should be read as indicative.

7.4 Strengths and limitations

Strengths

- The network matches or slightly exceeds the best classical model (test accuracy 98.25%, ROC-AUC 0.9931), with no false positives on the test set.
- Overfitting was demonstrated, diagnosed from the learning curves and controlled with standard, well-justified techniques.
- Model selection, evaluation and comparison were done without touching the test set, and under identical protocols for both models.
- The model is small (1,537 parameters, about 6 KB) and trains in seconds, so it is cheap to retrain and deploy.

Limitations

- The advantage over Logistic Regression is within random variation, so the extra complexity is not clearly justified on this dataset alone.
- The test set is small and comes from a single source; performance on data from other hospitals or scanners is unknown.
- Two malignant cases are missed with no reliable warning, which matters because false negatives are the costlier error in a medical setting.
- A neural network is harder to interpret than Logistic Regression, whose coefficients can be read directly.
- The hyperparameter search was small, and the learning rate and batch size were not tuned.

8. Possible Improvements and Implications

Improvements worth trying:

- **Lower the decision threshold** below 0.5 to favour recall, and measure how many extra false positives that costs.
- **Use repeated or nested cross-validation** so that model selection and performance estimation are both more stable on small data.
- **Ensemble several networks** (different seeds) and average them; deep ensembles typically give better uncertainty estimates than a single network with Monte Carlo dropout.
- **Calibrate the probabilities** (for example with temperature scaling) so that predicted probabilities can be trusted as risk estimates.
- **Validate externally** on data from a different source before drawing any clinical conclusion.

Implications. The main lesson is about method rather than about the winning model. A neural network can be trained reliably even on a few hundred samples, but only with

deliberate protection against overfitting, and its performance must be judged against a strong simple baseline and against its own run-to-run variation. On this dataset, a well-regularized network and Logistic Regression perform almost identically, which suggests the signal in the data is simple and strong, as Weeks 2 and 3 indicated. Deep learning would be expected to add real value only with richer inputs — such as the raw tumor images these measurements were derived from. In practice, either model would serve as a decision-support tool to prioritize cases for specialist review, not as a replacement for clinical judgement.

9. Project Files

- `deep_learning_model.py` — full code for this week (data preparation, overfitting demonstration, hyperparameter search, final training, evaluation, uncertainty and importance analysis)
- `fig_*.png` — the five figures used in this report
- `breast_cancer_nn.keras` — the trained final model (created when the script is run)
- GitHub repository: https://github.com/Develop-skills/data-cleaning-breast-cancer